

Group 1 — Dashboard Module Implementation Proposal

Blockchain-based EHR Platform: Architecture and Implementation Plan

Himanshu (24MCF1R16) — Team Lead Harsh Bhagat (24MCF1R14)
Dhruv Kumarawat (24MCF1R12)

September 2026

Contents

Group 1 — Dashboard Module Implementation Proposal	4
0. Scope of This Document	4
1. Current System Analysis	4
1.1 Existing Platform	4
1.2 Existing Peer / Role Mapping	5
2. Problem / Gap Analysis	6
2.1 Main Problem	6
2.2 Current UI Limitation	6
3. Requirements	7
3.1 Functional Requirements	7
4. Dashboard Information Architecture	10
5. High-Level Architecture	11
Key Principle	11
6. Agentic Peer Dashboard Architecture	11
6.1 Four Agentic Peer Views	12
6.2 Common Agent Dashboard Contract	12
7. Doctor Agent Dashboard	12
7.1 Purpose	12
7.2 Doctor Dashboard Wireframe	13
8. Nurse Agent Dashboard	13
8.1 Purpose	13
8.2 Wireframe	14
9. Pharmacy Agent Dashboard	14
9.1 Purpose	14
9.2 Wireframe	14
10. Lab Agent Dashboard	15
10.1 Purpose	15

10.2 Wireframe	15
11. Unified Agent Comparison View	16
12. Agent Execution Lifecycle	16
13. Agent Trace UI	16
14. Blockchain Network Dashboard	18
14.1 Network Topology	18
14.2 Network Cards	18
15. Transaction Dashboard	18
15.1 Metrics	18
15.2 Transaction Detail	19
16. Security Dashboard	19
17. IPFS Dashboard	19
18. Real-Time Event Architecture	20
18.1 Event Types	20
19. REST API Requirements	21
19.1 Overview	21
19.2 Network	21
19.3 Transactions	21
19.4 Agents	21
19.5 Security	22
19.6 IPFS	22
19.7 Events	22
20. Data Model	22
Agent	22
AgentExecution	22
Event	23
21. Storage Strategy	23
22. Frontend Architecture	23
23. Component Reuse Across Four Agents	24
24. Role-Based Dashboard Visibility	25
25. Metrics Specification	27
Network Metrics	27
Transaction Metrics	27
Agent Metrics	27
Storage Metrics	28
26. Alert Rules	28
27. Failure Handling	28
Peer Failure	28
Agent Failure	29
AI Failure	29

IPFS Failure	29
28. Security Design	30
29. Implementation Phases	30
Phase 1 — Dashboard Foundation	30
Phase 2 — Network Monitoring	31
Phase 3 — Four Agentic Peer Dashboards	31
Phase 4 — Agent Execution Trace	31
Phase 5 — Real-Time Events	31
Phase 6 — Security + IPFS	32
Phase 7 — Alerts + Polish	32
30. Testing Plan	32
Unit Tests	32
Integration Tests	32
Failure Tests	33
Authorization Tests	33
31. Demo Plan	33
Demo 1 — Overall Network	33
Demo 2 — Four Agentic Peers	33
Demo 3 — Generate a Lab Event	34
Demo 4 — Trace	34
Demo 5 — Failure	34
32. MVP vs Recommended vs Future	35
Architecture A — MVP	35
Architecture B — Recommended	35
Architecture C — Future	35
33. Cross-Module Integration	36
34. Important Design Decisions	37
Decision 1 — Dashboard is Observability, Not EHR Storage	37
Decision 2 — Four Agents Are First-Class Entities	37
Decision 3 — Common Agent Interface	37
Decision 4 — Role-Specific Metrics	38
Decision 5 — Real-Time First	38
Decision 6 — No Hard-Coded Physical Topology	38
35. Final Deliverables for Group 1	38
36. Final Implementation Checklist	39
Dashboard Core	39
Four Agentic Peers	39
Backend	39
Real-Time	39
Security	40
Testing	40
37. Expected Final Outcome	40

Group 1 — Dashboard Module Implementation Proposal

Project: Blockchain-based Electronic Health Record (EHR) Platform

Assigned Team: Group 1 — Dashboard Module

Team Lead: Himanshu (24MCF1R16)

Members: Harsh Bhagat (24MCF1R14), Dhruv Kumarawat (24MCF1R12)

Document Type: Proposed Architecture + Detailed Implementation Plan

Status: Implementation-oriented proposal

0. Scope of This Document

This document defines the **Dashboard Module** that Group 1 is responsible for implementing.

The dashboard is not treated as a generic admin page. Its purpose is to provide a unified observability layer over the existing EHR platform and, importantly, provide **role/peer-specific operational dashboards** for the four clinical agentic peers:

- **Doctor Peer / Doctor Agent Dashboard**
- **Nurse Peer / Nurse Agent Dashboard**
- **Pharmacy Peer / Pharmacy Agent Dashboard**
- **Lab Peer / Lab Agent Dashboard**

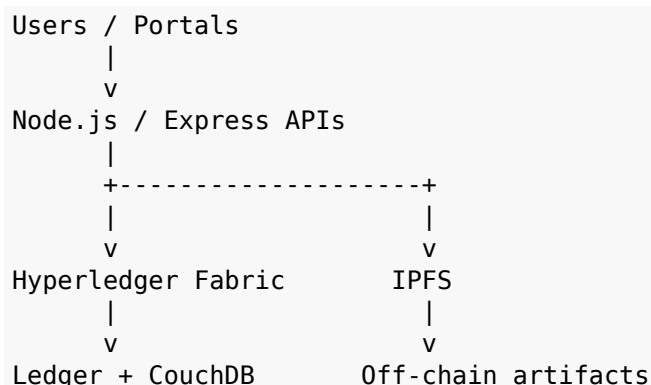
The dashboard should consume existing backend/Fabric/IPFS information rather than rewriting clinical workflows owned by other teams.

The architecture below is grounded in the supplied EHR vision and current project structure. The EHR vision explicitly assigns the dashboard team responsibility for monitoring network health, peer health, orderers, channels, block height, transaction metrics, agent health/execution, AI latency/failures, security events, consent events, IPFS health, storage, real-time events and alerts.

1. Current System Analysis

1.1 Existing Platform

The supplied project is a Hyperledger Fabric based EHR platform with:



The current deployment uses Fabric peers, CouchDB, Docker Swarm and an API layer. The project material also describes multiple role-specific APIs and role-to-peer mappings.

The current implementation already contains role-specific clinical workflows. Examples include:

- Doctor dashboard/workflow
- Nurse workflow
- Pharmacist workflow
- Lab workflow
- Patient portal
- Manager/admin portal

The dashboard module should therefore **observe and summarize these workflows**, not duplicate them.

1.2 Existing Peer / Role Mapping

The supplied project currently maps roles across peers/APIs. The deployment/status material identifies mappings such as:

```

Reception / Administration
    |
    v
Hospital peer / peer0

Doctor
    |
    v
Hospital peer / peer1

Nurse + Pharmacist + Medical Records
    |
    v
Hospital peer / peer2

Lab roles
    |
    v
Diagnostic peer

Provider / Insurance roles
    |
    v
Provider peer

```

The exact runtime topology may evolve as the agentic architecture is implemented. Therefore, the dashboard should use a **logical role/agent identity** separately from a hard-coded machine IP.

Important design rule

```

Logical Agent Identity
    |
    +--> Doctor Agent
    +--> Nurse Agent
    +--> Pharmacy Agent
    +--> Lab Agent
    |
    v

```

Physical Peer / API / Container

This prevents the dashboard from breaking when peer placement changes.

2. Problem / Gap Analysis

2.1 Main Problem

The existing EHR has many distributed components:

```
Fabric
├── Peers
├── Orderer
├── Channels
├── Chaincode
└── CouchDB

Clinical workflows
├── Doctor
├── Nurse
├── Lab
└── Pharmacy

AI
├── Agent execution
├── AI latency
├── AI failures
└── Provenance

Storage
└── IPFS

Security
├── Consent
├── Access denial
├── VCAP
└── Audit events
```

Without a unified dashboard, an operator has to inspect different portals, logs and services individually.

2.2 Current UI Limitation

The supplied project explicitly identifies manual page refresh as a limitation and proposes chaincode events with WebSocket/SSE to push updates to the frontend.

Therefore:

```
CURRENT

Frontend
|
| polling / manual refresh
```

```
  v
Backend
  |
  v
Fabric / IPFS
```

should evolve toward:

PROPOSED

```
Fabric / IPFS / Agent Services
      |
      v
    Event Collector
      |
      v
Dashboard Backend
      |
    WebSocket/SSE
      |
      v
    React Dashboard
```

3. Requirements

3.1 Functional Requirements

The dashboard must provide:

FR-01 — Overall System Status

Show:

- Organizations
- Peers
- Orderers
- Channels
- Chaincodes
- Current block height
- Transaction activity

The EHR vision explicitly lists these as Blockchain Network dashboard information.

FR-02 — Peer Health

For each relevant peer show:

- Online/offline
- Uptime
- Block height
- Ledger synchronization status
- CPU
- Memory

- Disk
- CouchDB status
- Peer connectivity
- Endorsement failures

FR-03 — Transaction Monitoring

Show:

- Transactions/minute
- Successful transactions
- Failed transactions
- MVCC conflicts
- Endorsement failures
- Average latency

These metrics are explicitly proposed in the supplied EHR vision.

FR-04 — Four Agentic Peer Dashboards

The dashboard must have dedicated operational views for:

Doctor Agent
Nurse Agent
Pharmacy Agent
Lab Agent

Each view must show:

- Agent/peer health
- Current status
- Requests/jobs
- Completed jobs
- Failed jobs
- Average latency
- Recent executions
- Relevant EHR events
- Relevant blockchain transactions
- AI/model status where applicable
- Recent errors
- Security/access events relevant to that role

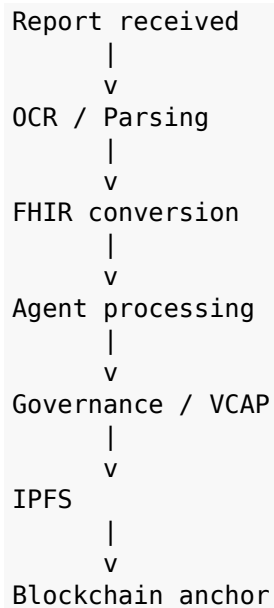
FR-05 — Agent Execution History

The EHR vision proposes execution history fields including:

Execution ID
Agent
Patient
Input CID
Output CID
Model
Duration
Status
VCAP

FR-06 — Agent Trace

The dashboard should visualize an AI/EHR execution as a pipeline:



The supplied architecture already describes this style of trace.

FR-07 — Security Monitoring

Show:

- Unauthorized requests
- Access denials
- Consent revocations
- Expired grants
- PDC violations
- VCAP verification failures
- Recent security events

These are directly specified in the EHR vision.

FR-08 — IPFS Monitoring

Show:

- IPFS node availability
- Gateway availability
- Artifact count
- Storage usage
- Recent upload failures
- CID resolution failures

FR-09 — Real-Time Events

Dashboard state should update without requiring a full page reload.

FR-10 — Alerts

The dashboard should identify:

CRITICAL

Peer down
Orderer unavailable
IPFS unavailable
Repeated transaction failures

WARNING

High latency
Ledger lag
High storage utilization
Increasing MVCC conflicts

INFO

New transaction
New AI execution
Consent changed
Artifact anchored

4. Dashboard Information Architecture

The proposed top-level structure is:

Dashboard

- 1. Executive Overview
- 2. Blockchain Network
 - Organizations
 - Peers
 - Orderers
 - Channels
 - Chaincodes
- 3. Agentic Peers
 - Doctor Agent
 - Nurse Agent
 - Pharmacy Agent
 - Lab Agent
- 4. Transactions
- 5. Security & Consent
- 6. Patient / EHR Metrics
- 7. IPFS
- 8. System Logs / Events

This follows the page structure proposed in the supplied EHR vision while making the four agentic peers a first-class dashboard section.

5. High-Level Architecture

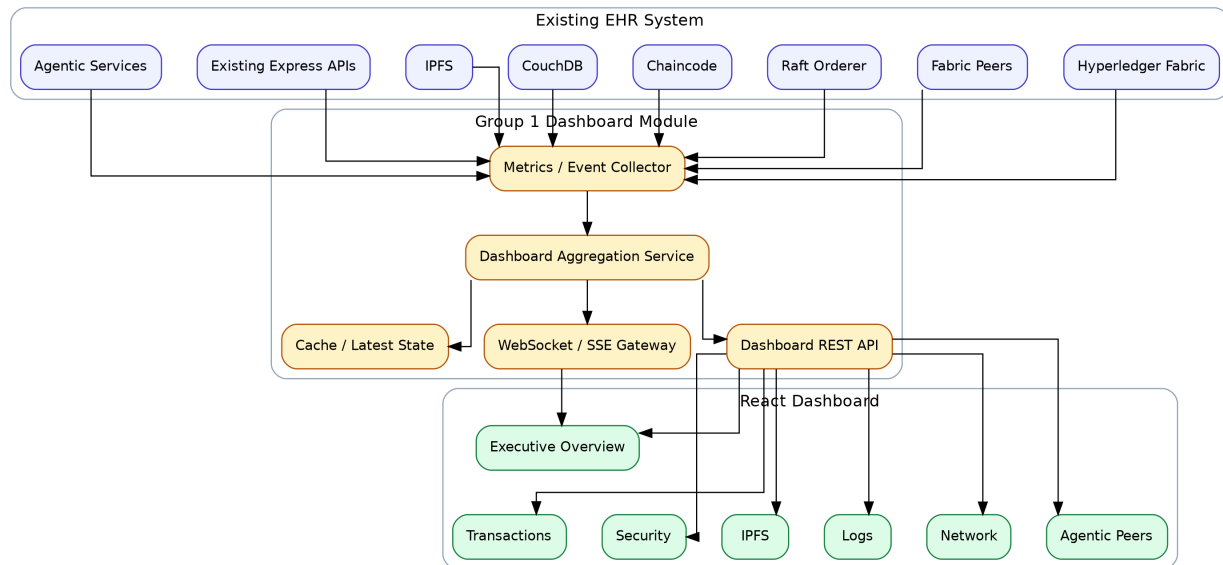
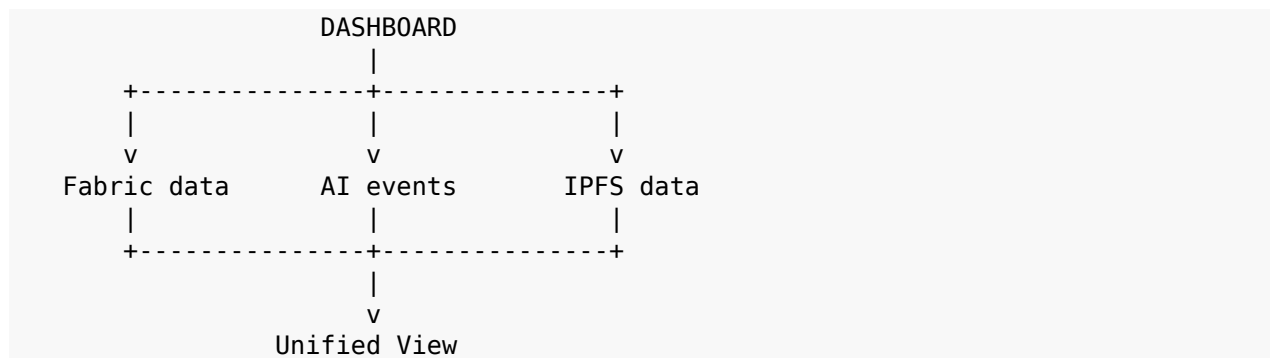


Figure 1: High-Level Dashboard Architecture

Key Principle

The dashboard should be an **observability layer**:

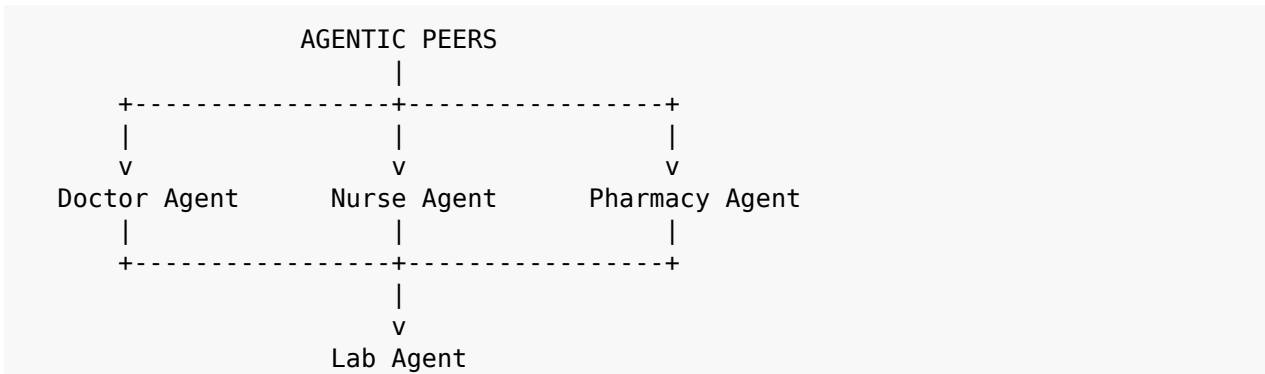


It should not become the owner of clinical records.

6. Agentic Peer Dashboard Architecture

This is the most important addition to the Group 1 implementation.

6.1 Four Agentic Peer Views



Each peer gets a dashboard card and a detailed page.

6.2 Common Agent Dashboard Contract

Every agent should expose the same basic observability model:

```
{
  "agentId": "doctor-agent-01",
  "agentType": "DOCTOR",
  "peerId": "peer1",
  "status": "ONLINE",
  "requests": 842,
  "successfulExecutions": 817,
  "failedExecutions": 25,
  "successRate": 97.03,
  "averageLatencyMs": 3800,
  "lastExecutionAt": "timestamp",
  "model": "configured-model",
  "modelVersion": "version",
  "vcapVerification": "PASS"
}
```

The exact model/provider should remain configuration-driven rather than being hard-coded into the dashboard.

7. Doctor Agent Dashboard

7.1 Purpose

The Doctor Agent dashboard monitors:

- Doctor peer availability
- Clinical workflow activity
- Patient/EHR access events
- Clinical AI requests
- Decision-support executions
- Lab-result related events
- Diagnosis/prescription workflow events
- Consent failures

- AI provenance/VCAP status

The current project contains a Doctor Dashboard for clinical management including diagnosis, prescriptions and lab orders.

7.2 Doctor Dashboard Wireframe

8. Nurse Agent Dashboard

8.1 Purpose

The Nurse Agent dashboard monitors:

- Nurse peer health
- Assigned patient/visit activity
- Vital recording events
- Nursing workflow transitions
- AI assistance requests
- Failed transactions
- Consent/access failures
- Recent patient-care events

The supplied chaincode includes a RecordVitals workflow under the clinical contract, confirming that nurse activity is a distinct clinical operation.

8.2 Wireframe

<

9. Pharmacy Agent Dashboard

9.1 Purpose

The Pharmacy Agent dashboard monitors:

- Pharmacy peer availability
- Medication-related transactions
- Prescription processing
- Medication dispensing events
- Inventory-related activity
- Failed transactions
- AI requests, if enabled
- Security/access events

The existing project identifies pharmacist roles and medication-dispensing workflow events.

9.2 Wireframe

+-----+			
PHARMACY AGENT			● ONLINE
Peer: pharmacy-peer-01	Agent: pharmacy-agent-01		
+-----+			
+-----+			

Prescriptions	Dispensed	Failed Txns	Avg Latency
64	58	2	1.4 sec

+-----+-----+-----+-----+			
PHARMACY ACTIVITY			
10:02 Prescription received			
10:03 Medication availability checked			
10:04 Medication dispensed			
10:09 Prescription rejected			
+-----+-----+-----+-----+			
INVENTORY / TRANSACTION HEALTH			
Low stock items: 5 Transaction success: 98.4%			
+-----+-----+-----+-----+			

10. Lab Agent Dashboard

10.1 Purpose

The Lab Agent dashboard monitors:

- Lab peer health
- Incoming lab reports
- Report processing
- OCR/parsing activity
- AI abnormality triage
- Lab result publication
- IPFS artifact generation
- Blockchain anchoring
- VCAP/provenance status
- Failed processing

The supplied architecture describes a Lab AI module responsible for report ingestion, normalization, IPFS storage and first-pass Gemini analysis.

10.2 Wireframe

LAB AGENT

Peer: diagnostic-peer-01

● ONLINE

Agent: lab-agent-01

Reports Today	AI Analyses	Success Rate	Avg Latency
128	117	98.2%	2.1 sec

LAB PROCESSING PIPELINE					
Reports → Parse/OCR → AI Triage → IPFS → VCAP → Fabric					
128	126	117	117	116	116
+-----+					
+-----+					
RECENT EXECUTIONS					
Report	AI Status	IPFS	VCAP	Fabric	
LAB-991	SUCCESS	PASS	PASS	ANCHORED	
LAB-992	SUCCESS	PASS	PASS	ANCHORED	
LAB-993	FAILED	--	--	--	
+-----+					

11. Unified Agent Comparison View

The executive dashboard should show all four agents together.

+-----+					
AGENTIC PEER HEALTH					
+-----+					
Agent	Peer	Status	Requests	Success	Latency
Doctor	peer1	ONLINE	842	97.8%	2.8s
Nurse	peer2	ONLINE	421	99.1%	1.9s
Pharmacy	pharmacy	ONLINE	310	98.4%	1.4s
Lab	diagnostic	ONLINE	1284	98.7%	2.1s
+-----+					

The EHR vision already gives the concept of an Agent Health section with online state, request count, success rate and latency. Group 1 should generalize that design across the four required agentic peers.

12. Agent Execution Lifecycle

13. Agent Trace UI

For an individual execution:

Execution #AI-1234

```
[1] INPUT RECEIVED
    |
    v
[2] PROCESSING
    |
    v
[3] AI INFERENCE
```

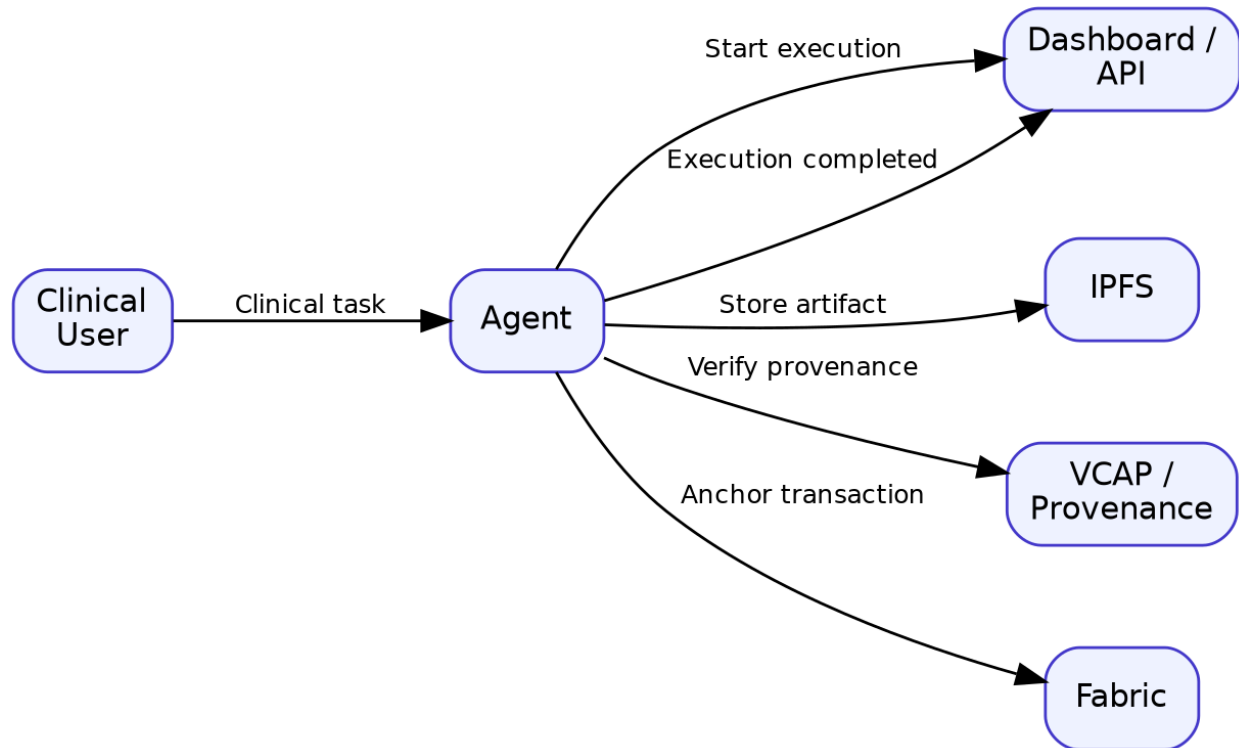



Figure 2: Agent Execution Sequence Flow

```

|
v
[4] OUTPUT GENERATED
|
v
[5] VCAP CREATED
|
v
[6] IPFS STORED
|
v
[7] FABRIC ANCHORED
  
```

Each stage should expose:

```

timestamp
duration
status
transaction ID / CID where applicable
error message if failed
  
```

14. Blockchain Network Dashboard

14.1 Network Topology

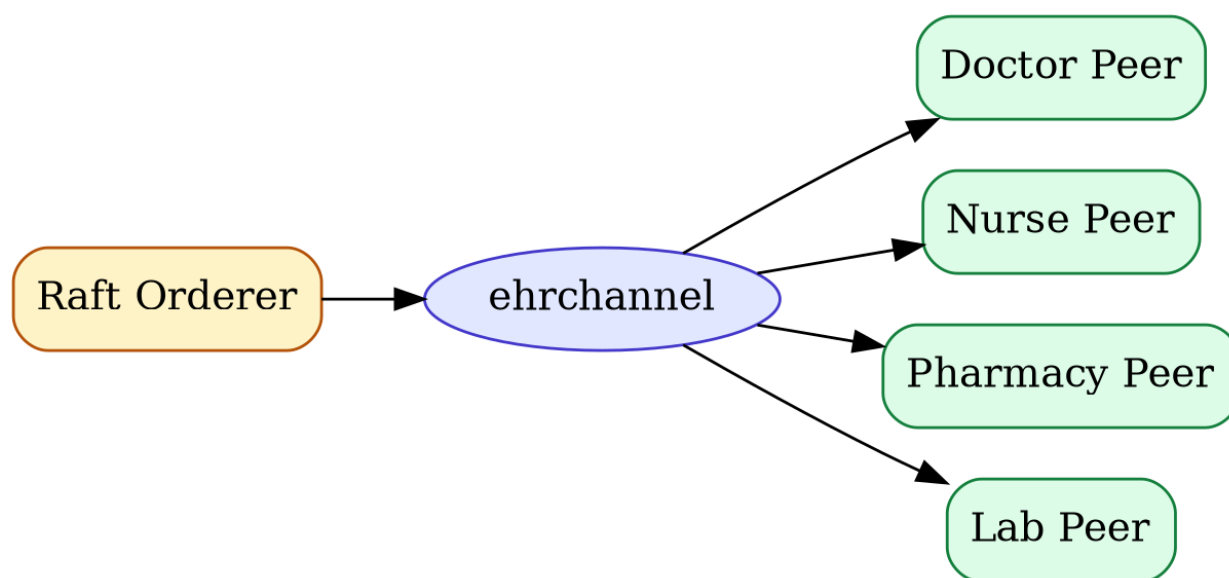


Figure 3: Fabric Channel and Peer Topology

The physical deployment may use a different number or placement of peers. The dashboard should therefore discover/report the actual configured topology instead of assuming that all four logical agents are necessarily separate physical machines.

14.2 Network Cards

Organizations	5
Peers	4+
Orderers	1+
Channels	1
Chaincodes	N
Block Height	12,481
Tx/min	42

15. Transaction Dashboard

15.1 Metrics

Transactions/min
Successful transactions
Failed transactions
MVCC conflicts
Endorsement failures
Average latency

These metrics are explicitly required in the supplied vision.

15.2 Transaction Detail

```
Transaction ID
Timestamp
Submitting role
Organization
Peer
Chaincode
Function
Status
Endorsement status
Commit status
Latency
Error code
Patient reference (only where permitted)
```

Patient identifiers should not be unnecessarily exposed in a global dashboard.

16. Security Dashboard

```
+-----+
| SECURITY                                     |
+-----+
| Unauthorized Requests          23          |
| Access Denials                 7           |
| Consent Revocations           12          |
| Expired Grants                 31          |
| PDC Violations                 0           |
| VCAP Verification Failures     2           |
+-----+
```

Recent events:

```
[12:42] Doctor requested EHR
[12:43] Patient granted access
[12:47] AI provenance verified
[13:02] Pharmacy access denied
[13:10] Patient revoked access
```

The above categories are directly aligned with the supplied EHR vision.

17. IPFS Dashboard

```
+-----+
| IPFS HEALTH                                     |
+-----+
| Node Status          ● ONLINE                |
```

Gateway Status	● ONLINE	
Storage Used	8.4 GB	
Artifacts	12,842	
Upload Failures	3	
CID Resolution Failures	1	
+-----+-----+		

The dashboard should monitor availability and metadata rather than expose sensitive clinical content.

18. Real-Time Event Architecture

The current project proposes `stub.setEvent` plus WebSocket/SSE for real-time chaincode-driven updates.

Recommended flow:

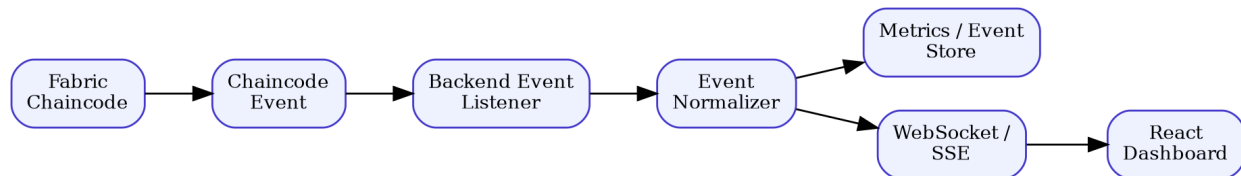


Figure 4: Real-Time Event Propagation Flow

18.1 Event Types

Use a common event envelope:

```

{
  "eventId": "evt-123",
  "eventType": "AGENT_EXECUTION_COMPLETED",
  "timestamp": "2026-09-15T10:20:00Z",
  "source": "doctor-agent",
  "peerId": "peer1",
  "severity": "INFO",
  "payload": {}
}

```

Suggested event types:

```

PEER_STATUS_CHANGED
BLOCK_HEIGHT_CHANGED
TRANSACTION_SUBMITTED
TRANSACTION_COMMITTED
TRANSACTION_FAILED
MVCC_CONFLICT
ENDORSEMENT_FAILED

AGENT_STARTED
AGENT_EXECUTION_STARTED
AGENT_EXECUTION_COMPLETED

```

```
AGENT_EXECUTION_FAILED  
  
CONSENT_GRANTED  
CONSENT_REVOKED  
ACCESS_DENIED  
VCAP_VERIFICATION_FAILED  
  
IPFS_UPLOAD_COMPLETED  
IPFS_UPLOAD_FAILED  
IPFS_NODE_DOWN
```

19. REST API Requirements

The dashboard backend should expose read-only aggregation APIs.

19.1 Overview

```
GET /api/dashboard/overview
```

Response:

```
{  
  "network": {},  
  "transactions": {},  
  "agents": {},  
  "security": {},  
  "ipfs": {}  
}
```

19.2 Network

```
GET /api/dashboard/network  
GET /api/dashboard/peers  
GET /api/dashboard/orderers  
GET /api/dashboard/channels
```

19.3 Transactions

```
GET /api/dashboard/transactions  
GET /api/dashboard/transactions/metrics
```

19.4 Agents

```
GET /api/dashboard/agents  
GET /api/dashboard/agents/:agentId  
GET /api/dashboard/agents/:agentId/executions  
GET /api/dashboard/agents/:agentId/metrics  
GET /api/dashboard/executions/:executionId
```

19.5 Security

```
GET /api/dashboard/security
GET /api/dashboard/security/events
```

19.6 IPFS

```
GET /api/dashboard/ipfs
```

19.7 Events

```
WebSocket /ws/dashboard
```

or

```
GET /api/dashboard/events
```

using SSE.

20. Data Model

The dashboard should maintain **observability data**, not duplicate the complete EHR ledger.

Agent

```
Agent
├── agentId
├── agentType
├── peerId
├── organization
├── status
├── model
├── modelVersion
├── lastHeartbeat
└── createdAt
```

AgentExecution

```
AgentExecution
├── executionId
├── agentId
├── patientRef (access-controlled)
├── inputCid
├── outputCid
├── model
├── duration
├── status
├── vcapStatus
└── transactionId
```

```

├─ startedAt
├─ completedAt

```

Event

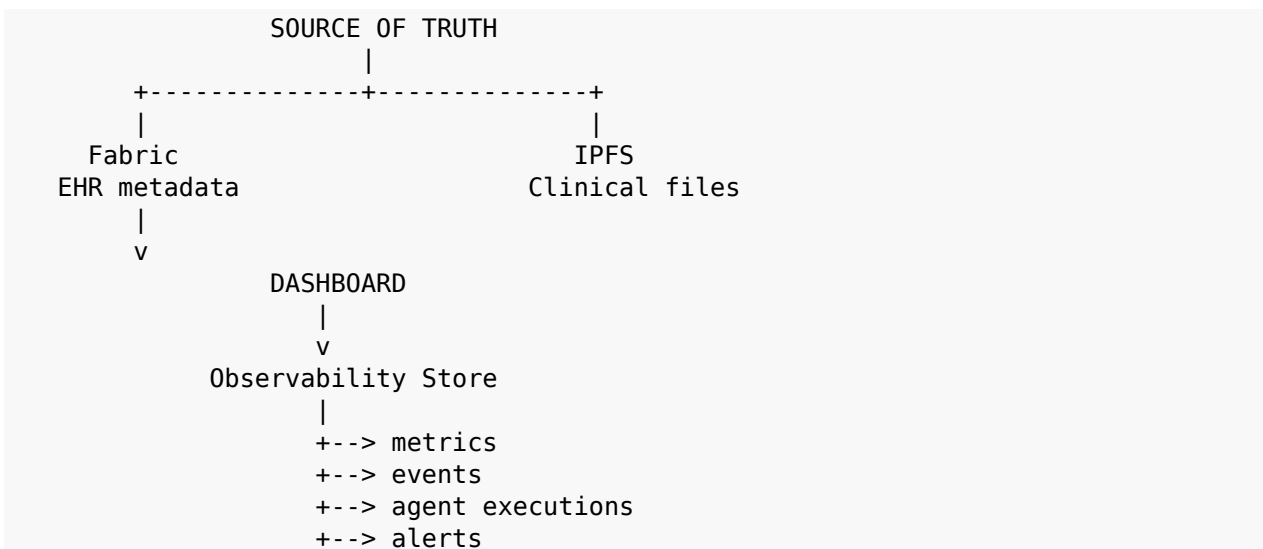
```

DashboardEvent
├─ eventId
├─ eventType
├─ source
├─ peerId
├─ severity
├─ timestamp
├─ payload

```

21. Storage Strategy

Use a separation between source-of-truth data and dashboard data:



Do **not** make the dashboard database the authoritative EHR database.

22. Frontend Architecture

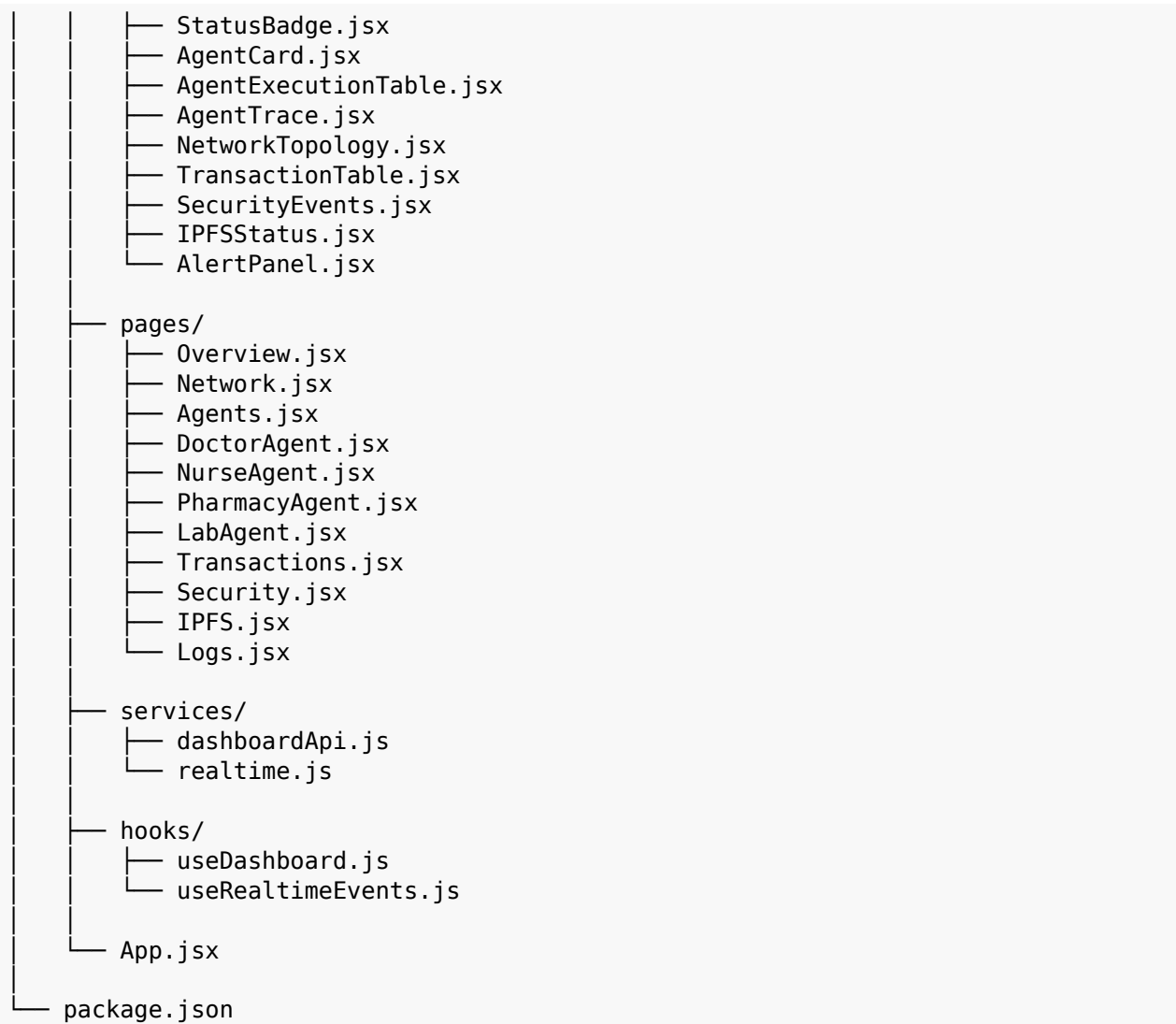
The supplied project already uses React/Vite for its frontend applications. The dashboard should remain consistent with that technology.

Recommended structure:

```

dashboard/
├─ src/
│   └─ components/
│       └─ MetricCard.jsx

```



23. Component Reuse Across Four Agents

Do not build four completely separate dashboard implementations.

Build one reusable component:

AgentDashboard

```
|
+---- configuration ---- Doctor
|
+---- configuration ---- Nurse
|
+---- configuration ---- Pharmacy
|
+---- configuration ---- Lab
```

Example configuration:


```

const agentConfig = {
  DOCTOR: {
    title: "Doctor Agent",
    peer: "peer1",
    metrics: [
      "activeVisits",
      "aiRequests",
      "successRate",
      "averageLatency"
    ]
  },
  NURSE: {
    title: "Nurse Agent",
    peer: "peer2",
    metrics: [
      "activePatients",
      "vitalsRecorded",
      "aiRequests",
      "averageLatency"
    ]
  },
  PHARMACY: {
    title: "Pharmacy Agent",
    peer: "pharmacy-peer",
    metrics: [
      "prescriptions",
      "dispensed",
      "failedTransactions",
      "averageLatency"
    ]
  },
  LAB: {
    title: "Lab Agent",
    peer: "diagnostic-peer",
    metrics: [
      "reports",
      "aiAnalyses",
      "successRate",
      "averageLatency"
    ]
  }
};

```

This keeps the UI consistent while allowing each peer to show domain-specific metrics.

24. Role-Based Dashboard Visibility

The dashboard itself must respect authorization.

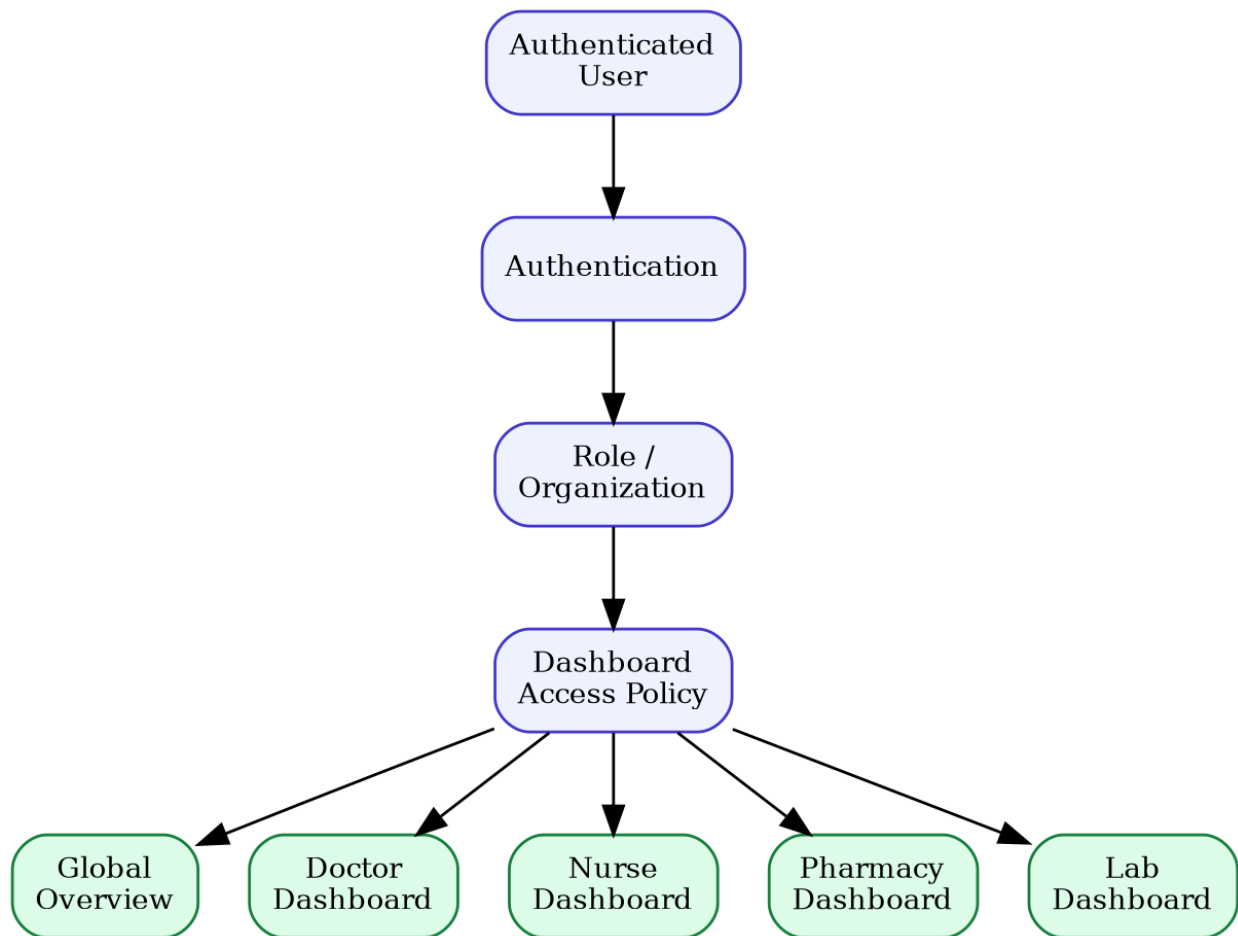


Figure 5: Role-Based Dashboard Access Control

Important:

Global dashboard

≠

Unlimited patient-data access

The dashboard should expose aggregate metrics where possible and restrict patient-level information according to existing access/consent rules.

The project already contains role-based identity and access-control helpers such as `requireRole`, `requireMSP`, and consent-aware EHR access checks.

25. Metrics Specification

Network Metrics

Metric	Meaning
Peer status	Online/offline
Block height	Latest committed block
Ledger lag	Difference from expected/latest height
Orderer status	Orderer availability
Channel status	Channel health
Chaincode status	Installed/committed/available

Transaction Metrics

Metric	Meaning
Tx/min	Transaction throughput
Success rate	Successful transactions / total
Failure rate	Failed transactions / total
MVCC conflicts	Version conflicts
Endorsement failures	Failed endorsements
Avg latency	Average transaction duration

Agent Metrics

Metric	Meaning
Agent status	Online/offline
Requests	Number of agent requests
Executions	Completed executions
Success rate	Successful executions
Failure rate	Failed executions
Avg latency	Average AI execution time
Model	Model currently used
VCAP status	Provenance verification
Last heartbeat	Last agent signal

Storage Metrics

Metric	Meaning
IPFS status	Node availability
Gateway status	Gateway availability
Storage used	Disk usage
Artifact count	Stored objects
Upload failures	Failed writes
CID failures	Failed retrievals

26. Alert Rules

Initial implementation:

```
IF peer.status == OFFLINE
    => CRITICAL

IF orderer.status == OFFLINE
    => CRITICAL

IF ipfs.status == OFFLINE
    => CRITICAL

IF agent.status == OFFLINE
    => CRITICAL

IF agent.averageLatency > threshold
    => WARNING

IF transaction.failureRate > threshold
    => WARNING

IF mvccConflictRate > threshold
    => WARNING

IF vcapVerificationFailure
    => HIGH

IF accessDeniedSpike
    => HIGH
```

Thresholds should be configurable.

27. Failure Handling

Peer Failure

```
Peer heartbeat fails
  |
  v
Event collector detects failure
  |
  v
Dashboard marks PEER DOWN
  |
  v
CRITICAL alert
```

Agent Failure

```
Agent heartbeat missing
  |
  v
Agent marked OFFLINE
  |
  v
Pending executions marked UNKNOWN
  |
  v
Dashboard displays degraded state
```

AI Failure

```
AI request
  |
  v
Inference fails
  |
  +--> execution = FAILED
  |
  +--> error recorded
  |
  +--> dashboard alert
```

The supplied project already describes fallback heuristic behavior when Gemini is unavailable for its implemented AI path. The dashboard should report whether an execution used the normal model or fallback mode rather than silently treating both as identical.

IPFS Failure

```
Artifact upload
  |
  X
  failure
  |
  v
Dashboard records failure
  |
```

↓
No false "anchored" status

28. Security Design

The dashboard backend must not bypass the project's existing identity and authorization model.

Required controls:

Authentication

↓

↓

Role / Organization

↓

↓

Dashboard authorization

↓

↓

Metric / event filtering

↓

↓

Patient-level access check

Sensitive data should not be placed in:

- browser local storage unnecessarily
- unrestricted global event streams
- logs
- metric labels
- public dashboard URLs

Prefer:

Aggregate metric

over

Raw patient record

29. Implementation Phases

Phase 1 — Dashboard Foundation

Goal: Get the shell running.

Implement:

- React/Vite dashboard
- Routing
- Navigation
- Metric cards
- Status badges
- Loading/error states

- Dashboard REST client

Deliverable:

```
Overview
Network
Agents
Transactions
Security
IPFS
Logs
```

Phase 2 — Network Monitoring

Implement:

- Peer status
- Block height
- Orderer status
- Channel status
- Transaction metrics

Phase 3 — Four Agentic Peer Dashboards

Implement:

```
Doctor Agent
Nurse Agent
Pharmacy Agent
Lab Agent
```

with:

- Common AgentDashboard component
- Role-specific metrics
- Execution tables
- Agent health
- Error panel

Phase 4 — Agent Execution Trace

Implement:

```
Execution
|
+--> input
+--> processing
+--> model
+--> output
+--> VCAP
+--> IPFS
+--> Fabric
```

Phase 5 — Real-Time Events

Implement:

```
Chaincode events
  |
  v
Backend listener
  |
  v
WebSocket/SSE
  |
  v
React state update
```

Phase 6 — Security + IPFS

Implement:

- Security event feed
- Consent event feed
- VCAP failures
- IPFS health
- Storage metrics

Phase 7 — Alerts + Polish

Implement:

- Severity levels
 - Alert panel
 - Filtering
 - Time ranges
 - Agent comparison
 - Responsive UI
 - Error boundaries
-

30. Testing Plan

Unit Tests

Test:

```
metric calculations
status mapping
agent configuration
event normalization
alert rules
API response mapping
```

Integration Tests

Verify:

```
Fabric → event listener
Fabric → dashboard API
```



```
Agent → execution event
IPFS → dashboard
API → React
WebSocket/SSE → React
```

Failure Tests

Simulate:

```
peer down
orderer down
agent down
IPFS down
AI failure
transaction failure
MVCC conflict
VCAP failure
network interruption
```

Authorization Tests

Verify:

```
Doctor cannot see unauthorized pharmacy data
Nurse cannot access unrestricted global patient data
Pharmacy cannot see private clinical content
Lab cannot see unrelated private records
Unauthorized dashboard endpoint => 403
```

31. Demo Plan

The final Group 1 demo should tell one coherent story.

Demo 1 — Overall Network

Show:

```
Peers
Orderer
Channel
Block height
Transactions
```

Demo 2 — Four Agentic Peers

Show:

Doctor	● ONLINE
Nurse	● ONLINE
Pharmacy	● ONLINE
Lab	● ONLINE

Demo 3 — Generate a Lab Event

```
Lab report
  |
  v
Lab Agent
  |
  v
AI analysis
  |
  v
IPFS
  |
  v
VCAP
  |
  v
Fabric
```

Dashboard should update automatically.

Demo 4 — Trace

Open the execution and show:

```
Input
  ↓
AI
  ↓
Output
  ↓
VCAP
  ↓
IPFS
  ↓
Fabric
```

Demo 5 — Failure

Stop an agent/peer or simulate an error:

```
ONLINE
  ↓
OFFLINE
  ↓
Dashboard alert
```

This demonstrates that the dashboard is genuinely monitoring the system rather than displaying static values.

32. MVP vs Recommended vs Future

Architecture A — MVP

```
React Dashboard
  |
  v
Express Dashboard API
  |
  +--> Fabric queries
  +--> IPFS health
  +--> Agent health
```

Features:

- Overview
- Network
- Four agent cards
- Basic transaction metrics
- Basic logs

Architecture B — Recommended

```
Fabric / Agents / IPFS
  |
  v
Event Collector
  |
  v
Metrics + Event Store
  |
  +-----+-----+
  |         |         |
  | REST    | WS/SSE   |
  |         |         |
  +-----+-----+
  |
  v
React UI
```

Features:

- Real-time events
- Agent execution history
- Trace
- Alerts
- Security events
- IPFS metrics
- Role-based views

Architecture C — Future

Distributed Observability Platform

```

Fabric
Agents
IPFS
APIs
|
v
Event Bus
|
+--> Metrics
+--> Logs
+--> Traces
|
v
Observability Store
|
v
Dashboard

```

Possible future capabilities:

- historical time-series analytics
- distributed tracing
- anomaly detection
- predictive alerts
- cross-organization analytics

33. Cross-Module Integration

The dashboard is dependent on all major project pillars:

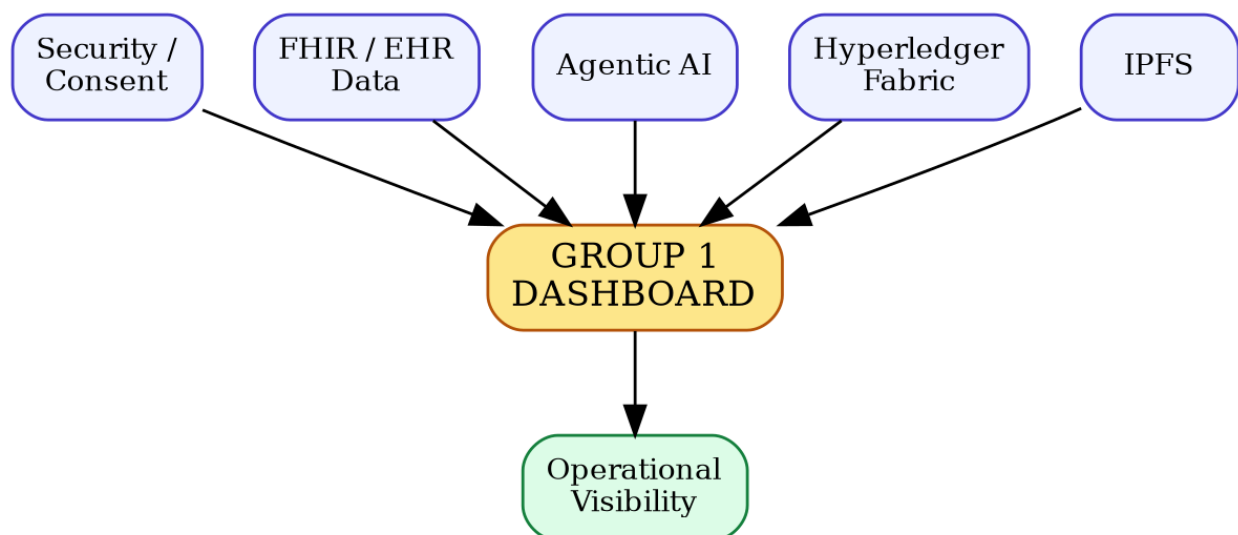


Figure 6: Cross-Module Integration Overview

Examples:

```

AI team
|
+--> Agent execution events
+--> Model metadata
+--> VCAP result
|
v
Dashboard

Security team
|
+--> Access events
+--> Consent events
+--> Violations
|
v
Dashboard

FHIR team
|
+--> Resource/event types
|
v
Dashboard

```

The supplied architecture explicitly describes AI outputs flowing through VCAP, IPFS and Fabric, with dashboard metrics as an output of that workflow.

34. Important Design Decisions

Decision 1 — Dashboard is Observability, Not EHR Storage

Fabric/IPFS = source of truth
Dashboard store = monitoring state

Decision 2 — Four Agents Are First-Class Entities

The dashboard must explicitly support:

Doctor Agent
Nurse Agent
Pharmacy Agent
Lab Agent

rather than only showing a generic “AI Agent” page.

Decision 3 — Common Agent Interface

All four agents should expose a common health/execution contract.

Decision 4 — Role-Specific Metrics

Common metrics:

```
status
requests
success rate
latency
failures
```

Domain metrics:

```
Doctor    → visits / clinical AI
Nurse     → patients / vitals
Pharmacy  → prescriptions / dispensing
Lab       → reports / AI triage
```

Decision 5 — Real-Time First

The dashboard should be designed around events, with REST used for initial state/history and WebSocket/SSE used for live changes.

This directly addresses the existing manual-refresh limitation identified in the project.

Decision 6 — No Hard-Coded Physical Topology

Use:

```
agentId
agentType
organization
peerId
```

and resolve the physical endpoint through configuration/service discovery.

35. Final Deliverables for Group 1

The Group 1 implementation should result in:

```
dashboard/
|
├── information-architecture.md
├── ui-design.md
├── metrics-specification.md
├── event-architecture.md
├── api-requirements.md
├── realtime-design.md
├── implementation-roadmap.md
|
├── frontend/
│   ├── components/
│   ├── pages/
│   ├── services/
│   └── hooks/
```

```
└─ backend/
    └─ routes/
    └─ services/
    └─ collectors/
    └─ events/
```

The supplied EHR vision explicitly identifies the documentation deliverables above for the dashboard team.

36. Final Implementation Checklist

Dashboard Core

- ☐ React dashboard shell
- ☐ Routing
- ☐ Overview
- ☐ Network page
- ☐ Transactions page
- ☐ Security page
- ☐ IPFS page
- ☐ Logs page

Four Agentic Peers

- ☐ Doctor Agent dashboard
- ☐ Nurse Agent dashboard
- ☐ Pharmacy Agent dashboard
- ☐ Lab Agent dashboard
- ☐ Common agent health component
- ☐ Role-specific metrics
- ☐ Execution history
- ☐ Execution trace

Backend

- ☐ Overview API
- ☐ Network API
- ☐ Transaction API
- ☐ Agent API
- ☐ Execution API
- ☐ Security API
- ☐ IPFS API

Real-Time

- ☐ Fabric event listener
- ☐ Event normalization
- ☐ WebSocket/SSE
- ☐ Frontend subscription
- ☐ Automatic state update

- ☐ Reconnection handling

Security

- ☐ Dashboard authentication
- ☐ Role-based authorization
- ☐ Organization filtering
- ☐ Patient-level access protection
- ☐ Sensitive-data minimization
- ☐ Security event monitoring

Testing

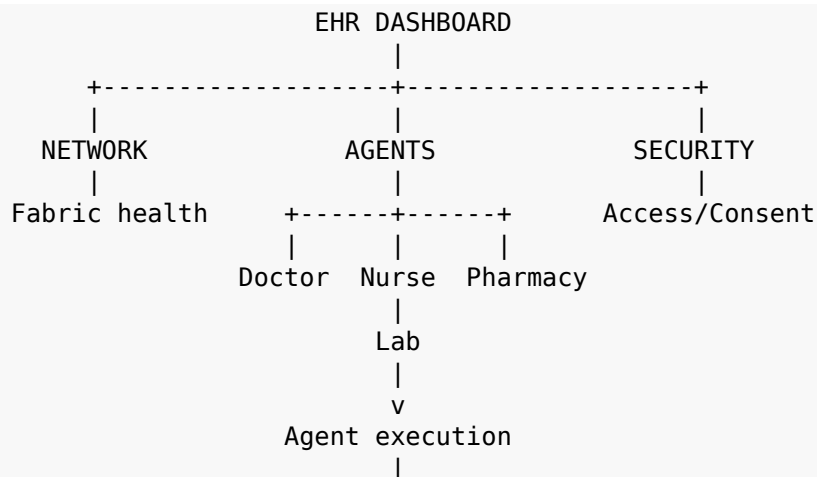
- ☐ Unit tests
- ☐ API integration tests
- ☐ Fabric integration tests
- ☐ Agent integration tests
- ☐ Real-time tests
- ☐ Failure tests
- ☐ Authorization tests

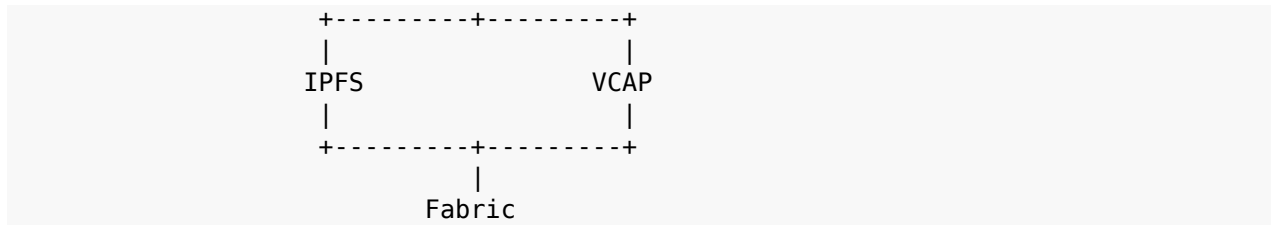
37. Expected Final Outcome

At completion, Group 1 should deliver a dashboard that answers five questions immediately:

1. Is the EHR blockchain network healthy?
2. Are the Doctor, Nurse, Pharmacy and Lab peers/agents healthy?
3. What transactions and AI executions are happening?
4. Are there security, consent, IPFS or AI-provenance problems?
5. If something failed, exactly where did it fail?

The final experience should look conceptually like:





Group 1's core contribution is therefore the observability and operational control plane for the complete EHR system, with the four agentic peers—Doctor, Nurse, Pharmacy and Lab—treated as first-class monitored entities.

This proposal deliberately distinguishes the **logical four agentic peers** from the current physical Fabric topology. The current project material contains multiple peer/role mappings; the implementation should make peer endpoints configurable rather than assuming one fixed machine layout.